Responsive Design, Media Queries & User Interface

A comprehensive technical guide to building adaptive, device-agnostic web experiences

Responsive Design

Fluid layouts that adapt to any screen size

Media Queries

Conditional CSS rules for breakpoint-driven styling

User Interface

Visual and interactive components that users engage with

1. Responsive Design

Responsive Design is a web design approach used to make websites automatically adapt their layout, content, images, and components according to the screen size and device being used.

A responsive website should provide a good user experience on mobile phones, tablets, laptops, desktops, and large screens without creating separate websites for each device.

Why Responsive Design is Important

- Websites are accessed from different screen sizes.
- Improves mobile and tablet usability.
- Prevents horizontal scrolling.
- Makes layouts flexible and adaptable.
- Improves accessibility and user experience.
- Reduces the need to maintain separate mobile and desktop websites.

Layout Example

Desktop layout:

```
-----------------------------------------------
| Logo | Home | About | Services | Contact    |
-----------------------------------------------
|      |                      |       |
| Side |      Main Content          | Side  |
| bar  |                      | bar   |
-----------------------------------------------
```

Mobile layout:

```
------------------------
| Logo                 |
------------------------
|                      |
|     Main Content     |
|                      |
------------------------
| Sidebar Content      |
------------------------
```

The content remains the same, but the layout changes according to available space.

2. Important Concepts of Responsive Design

Fluid Layout

A fluid layout uses relative units such as %, vw, and rem instead of relying entirely on fixed pixel values.

```
.container {
    width: 90%;
    margin: auto;
}
```

Instead of:

```
.container {
    width: 1200px;
}
```

Flexible Images

Images should generally be prevented from becoming wider than their containers.

```
img {
    max-width: 100%;
    height: auto;
}
```

Viewport

The viewport is the visible area of a webpage on a device. For responsive websites, the following meta tag is commonly used:

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

It tells the browser to use the device's actual width when displaying the webpage.

Relative Units

Common responsive units include:

%

Relative to the parent element

vw

Viewport width

vh

Viewport height

rem

Relative to root font size

em

Relative to the current element's font size

3. Media Queries

Media Queries are a CSS feature that allows developers to apply different styles depending on conditions such as screen width, height, orientation, or device characteristics. They are one of the most important tools for implementing responsive design.

Basic Syntax

```
@media (condition) {
    selector {
        property: value;
    }
}
```

Example

```
.card {
    width: 400px;
}

@media (max-width: 600px) {
    .card {
        width: 90%;
    }
}
```

How it works:

- Above 600px → card is 400px
- At or below 600px → card becomes 90% wide

4. Common Media Query Conditions

max-width

Applies styles when the viewport is equal to or **smaller** than the specified width.

```
@media (max-width: 768px) {
    body {
        background-color: lightblue;
    }
}
```

min-width

Applies styles when the viewport is equal to or **larger** than the specified width.

```
@media (min-width: 1024px) {
    .container {
        max-width: 1200px;
    }
}
```

Orientation

Change styles depending on whether the device is in portrait or landscape mode.

```
@media (orientation: landscape) {
 .container {
 display: flex;
 }
}
```

5. Breakpoints

A breakpoint is a screen size at which the layout changes to provide a better user experience.

Standard Breakpoint Structure

```
/* Mobile */
@media (max-width: 600px) {

}

/* Tablet */
@media (min-width: 601px) and (max-width: 1024px) {

}

/* Desktop */
@media (min-width: 1025px) {

}
```

Breakpoint Strategy

Developers should not blindly design for specific devices. Breakpoints should ideally be selected based on where the content or layout starts looking incorrect.

Mobile

max-width: 600px — single column, stacked elements

Tablet

601px–1024px — two columns, condensed navigation

Desktop

min-width: 1025px — full multi-column layout

6. Mobile-First Design

Mobile-first design means designing the website for smaller screens first and then progressively adding styles for larger screens.

Mobile-First Example

```
.container {
    width: 100%;
    padding: 15px;
}

@media (min-width: 768px) {
    .container {
        width: 80%;
        margin: auto;
    }
}
```

The basic CSS handles mobile devices, while the media query enhances the design for larger screens.

Mobile-First vs Desktop-First

Desktop-first:

```
Desktop
  ↓
Tablet
  ↓
Mobile
```

Mobile-first:

```
Mobile
  ↓
Tablet
  ↓
Desktop
```

Mobile-first development is useful because it encourages developers to prioritize the most important content and interactions.

7. Responsive Layout Using Flexbox

Flexbox can automatically adjust the arrangement of elements based on available screen space.

Flexbox + Media Query

```
.container {
    display: flex;
    gap: 20px;
}

@media (max-width: 700px) {
    .container {
        flex-direction: column;
    }
}
```

Desktop Layout

```
[ Card ] [ Card ] [ Card ]
```

Elements are arranged in a horizontal row using `flex-direction: row` (default).

Mobile Layout

```
[ Card ]

[ Card ]

[ Card ]
```

At or below 700px, `flex-direction: column` stacks elements vertically, preventing overflow and improving readability on narrow screens.

8. Responsive Layout Using CSS Grid

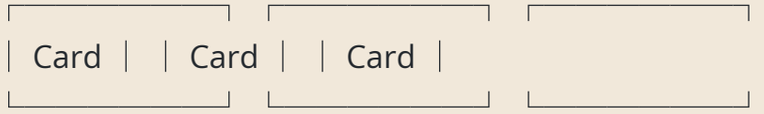
CSS Grid can also be combined with media queries to create powerful multi-column layouts that collapse gracefully on smaller screens.

Grid + Media Query

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}

@media (max-width: 768px) {
    .container {
        grid-template-columns: 1fr;
    }
}
```

Desktop (3 columns)



Mobile (1 column)



The `grid-template-columns` property switches from three equal columns to a single full-width column at the 768px breakpoint.

9. User Interface — UI

User Interface (UI) is the visual and interactive part of a website or application through which users interact with the system.

UI Components Include:

Controls

Buttons, Input fields, Dropdowns, Forms

Navigation

Navigation bars, Menus, Modals

Content

Cards, Icons, Images, Typography

Style

Colors, Spacing

E-Commerce UI Example

```
-------------------------------------------
| LOGO    Search...    Cart  Account    |
-------------------------------------------
| Home | Products | Categories | Offers    |
-------------------------------------------
|                                         |
|          Product Image                  |
|                                         |
|          Product Name                   |
|          ₹1,999                         |
|                                         |
|          [ Add to Cart ]                |
-------------------------------------------
```

All of these visual elements together form part of the User Interface.

10. Important UI Principles

Visual Hierarchy

Visual hierarchy determines which information attracts the user's attention first.

```
MAIN HEADING
    ↓
Supporting information
    ↓
Important details
    ↓
[ Call to Action ]
```

A heading might use a larger font, while secondary information uses a smaller font.

Consistency

Similar components should look and behave similarly.

```
[ Save ]  [ Submit ]  [ Continue ]
```

If all primary buttons use the same design, users can easily recognize them.

Spacing

Proper spacing improves readability and prevents the interface from feeling crowded.

```
.card {
    padding: 20px;
    margin: 20px;
}
```

Typography

Typography includes the design and arrangement of text. Important properties include: `font-family`, `font-size`, `font-weight`, `line-height`, `letter-spacing`, `text-align`

```
h1 {
    font-size: 36px;
    line-height: 1.2;
}

p {
    font-size: 16px;
    line-height: 1.6;
}
```

Color

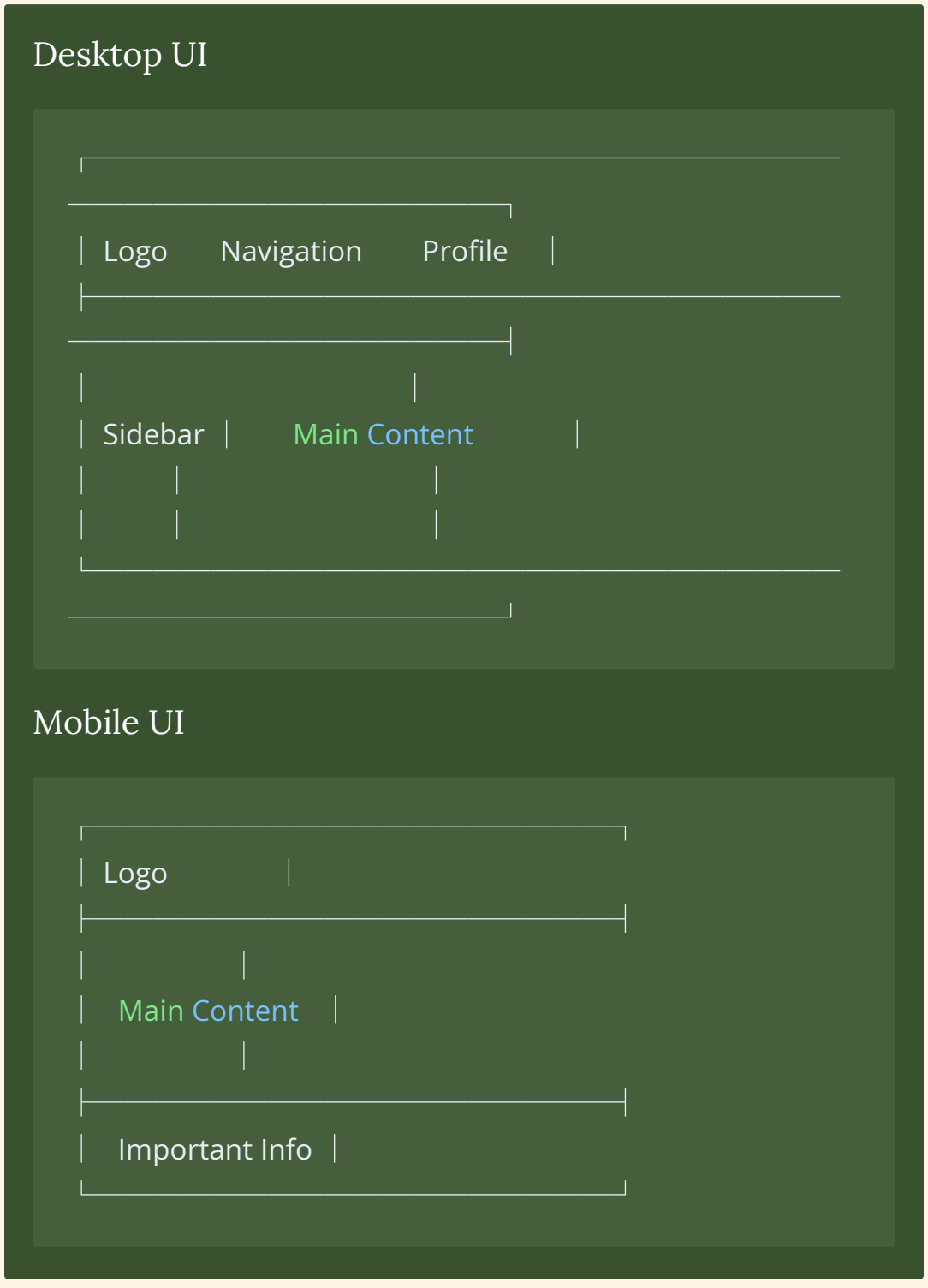
Colors should support readability and communicate meaning.

- **Green** → Success
- **Red** → Error / Warning
- **Blue** → Information / Action
- **Gray** → Secondary content

Color should not be the only way to communicate important information.

11. Responsive UI

A good UI should not only look attractive on a desktop but should also remain usable on smaller screens.



Responsive UI Adjustments

Responsive UI may require changing:

Navigation

Hamburger menus replace full nav bars

Columns & Layout

Number of columns, sidebar position, content order

Sizing & Spacing

Font sizes, image sizes, padding, button sizes

Visibility

Secondary content may be hidden on smaller screens

12. Responsive Navigation Example

This demonstrates how a navigation interface can change based on screen size.

HTML Structure

```
<nav class="navbar">
    <h2>MyWebsite</h2>

    <div class="links">
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Services</a>
        <a href="#">Contact</a>
    </div>
</nav>
```

CSS with Media Query

```
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 20px;
}

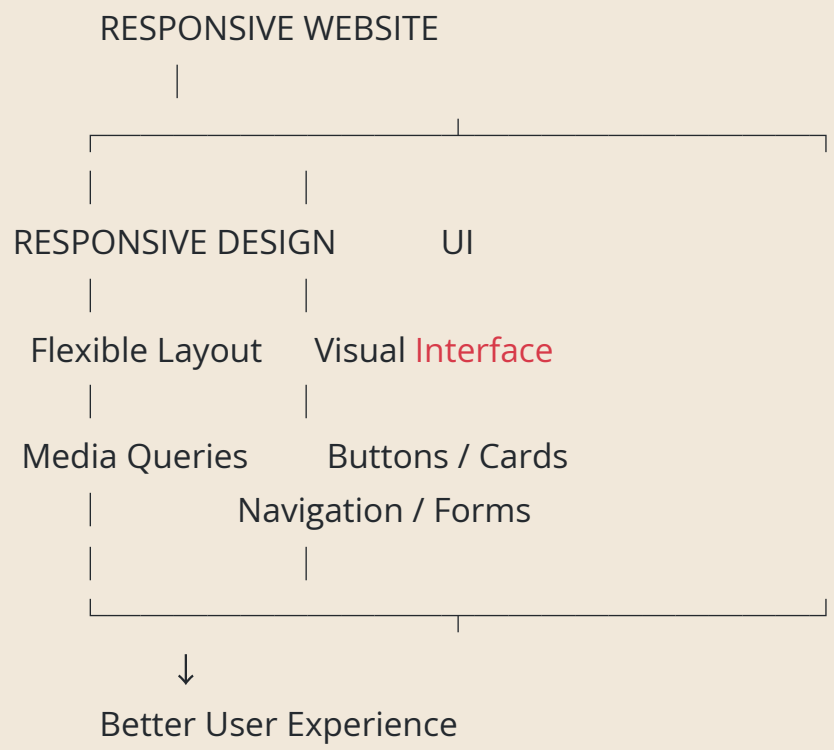
.links {
    display: flex;
    gap: 20px;
}

@media (max-width: 600px) {
    .links {
        display: none;
    }
}
```

On screens wider than 600px, all navigation links are displayed inline. On screens 600px or narrower, the links are hidden — typically replaced by a hamburger menu icon for mobile users.

13. Responsive Design + UI: Complete Concept

Students should understand that these three topics are connected:



Responsive Design

Determines how the layout adapts across different screen sizes and devices.

Media Queries

Provide conditional CSS rules to change the layout at defined breakpoints.

User Interface

Determines how the website looks and how users interact with it visually.

Classroom Practice Task

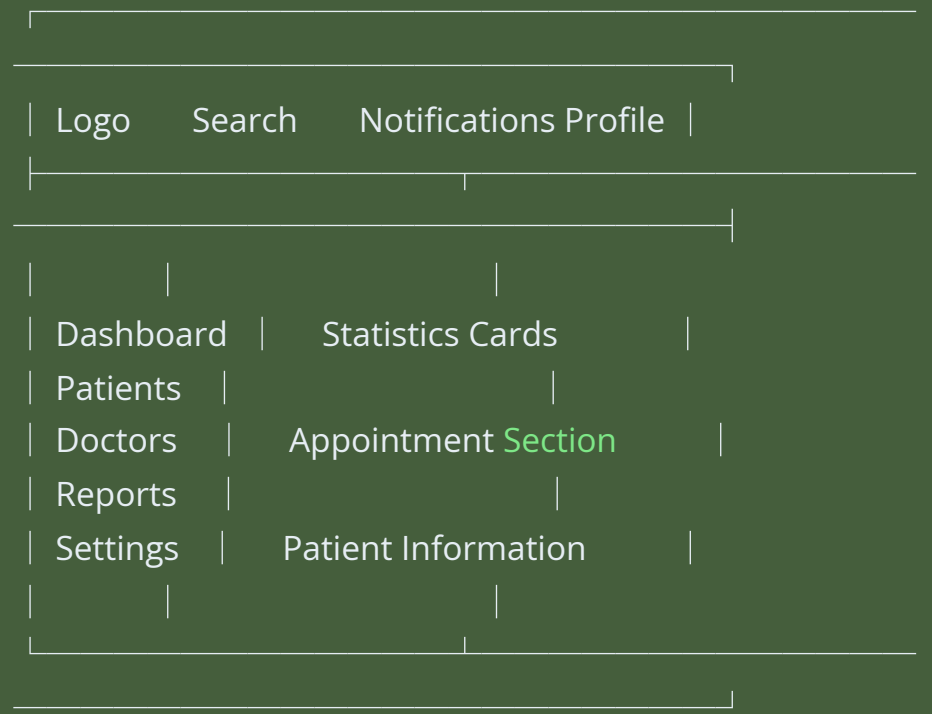
Task: Responsive Healthcare Dashboard

Create a responsive healthcare dashboard using HTML and CSS only.

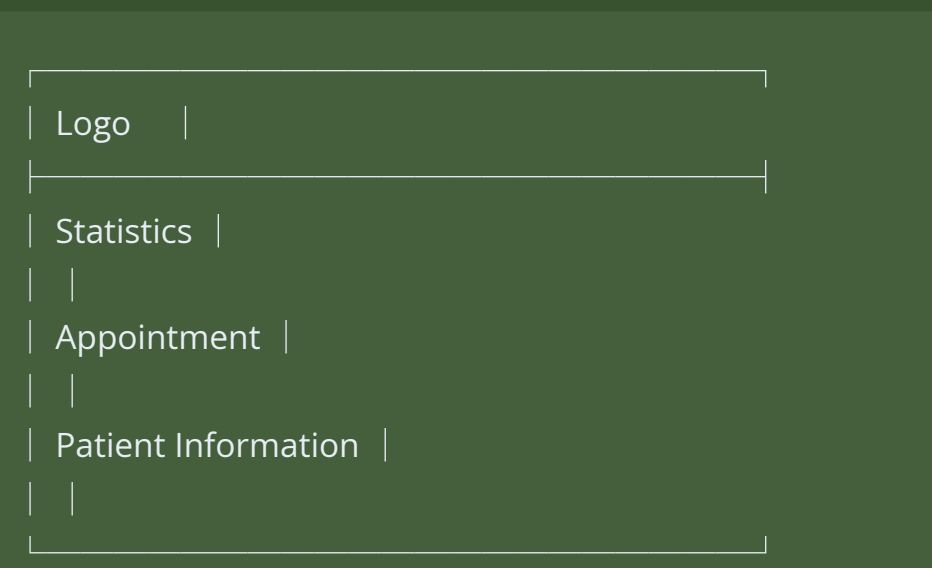
Required Components

- Fixed/sticky header
- Responsive navigation
- Sidebar
- Main dashboard
- Doctor/patient cards
- Appointment section
- Statistics cards
- Responsive images
- Buttons and form elements
- Media queries for mobile and tablet layouts

Desktop Layout



Mobile Layout



Learning Outcome

After completing this topic, students should be able to:

01

Explain responsive design

Understand the purpose and principles of responsive web design.

02

Create fluid layouts & use relative CSS units

Apply %, vw, vh, rem, and em units; understand the viewport meta tag.

03

Write media queries

Use max-width, min-width, and orientation conditions to apply breakpoint-driven styles.

04

Create mobile-first layouts

Design for small screens first, then progressively enhance for larger screens.

05

Build responsive navigation & images

Implement collapsible navbars and flexible images using CSS.

06

Adapt Flexbox and Grid layouts

Use display: flex and display: grid with media queries for responsive structure.

07

Design basic user interfaces

Apply visual hierarchy, consistency, spacing, typography, and color principles.

08

Evaluate cross-device usability

Create interfaces that work across mobile, tablet, and desktop screens, and assess visual consistency.